LiteScript assembly cheatsheet

The instructions of the LiteScript stack machine, written directly. Scripts are compiled into these instructions. **In this sheet, `; text` is an annotation, not syntax: the assembler has no comments.** Full reference: docs/reference/assembly.md.

Syntax

Line	`name argument` — one instruction per line, name and argument separated by the first space or tab
Blanks	leading spaces and `\r` ignored; an empty line does nothing but **counts as a line**
Line numbers	start at **1**; `jump-*`, `value.callback`, `try.begin` target a line; a line after the end = stop
Comments	**none**: any unknown text is an error
Case	instruction names are lower case, exact

Argument	Format
`<name>`	trimmed text, can't be empty (`a.b` allowed for `namespace`)
`<string>`	`value.string` : **rest of the line after the first space**, spaces kept (`value.string = ""`)
`<index>`	unsigned integer: `0`, `12`
`<line>`	unsigned integer ≥ 1
`<number>`	`42`, `-6`, `+12` (int64) · `2.5`, `1e3`, `2.0` (float) · no `inf` / `nan`
`<true|false>`	`true` or `false` only

String escapes (`<name>`, `<string>`): `\n \r \t \0 \\` , any other `\c` gives `c` .

Compile error	Cause
`Unknown assembly command`	unknown name, comment, typo
`Boolean argument expected`	`value.boolean yes`
`Unsigned integer argument expected`	`define.arg -1`, `jump-to 0`, missing line
`Number argument expected`	`value.number abc`, `inf`
`String argument expected`	`value.variable` without name

The machine

Part	Role
Value stack	`value.*` push, operators pop operands and push the result
Argument lists	`push.args` opens a list, `define.arg i` fills it, `operator.call` / `value.object` **consume** it
Frames	a call (or `push.nsp`) creates a local namespace; `return` / `pop.nsp` leave it
Return value	`define.return` stores it (doesn't leave), `return` leaves and pushes it for the caller
Try handlers	`try.begin` records the catch line; an error restores the stack and jumps there with the error value pushed
Namespace	`namespace` sets where `define.variable` declares outside functions
Program result	the last value on the stack (or the value given to `define.return` + `return` at top level)

Definitions & values

Stack effect: `-n` popped, `+n` pushed.

Instruction	Stack	Effect
`define.variable <name>`	−1	declare in the current function / namespace
`define.arg <index>`	−1	set argument `index` of the open list (holes = undefined)
`define.return`	−1	set the return value
`value.pop`	−1	discard
`value.copy`	+1	duplicate the top (**same** object)
`value.assign`	−1 +1	replace the top by a copy (what `=` does)
`value.undefined` / `value.null`	+1	
`value.boolean <true|false>`	+1	
`value.number <number>`	+1	
`value.string <string>`	+1	
`value.array`	+1	empty array
`value.class`	+1	empty class
`value.object`	−1 +1	`new` the popped class with the open list, calls the constructor
`value.callback <line>`	+1	function starting at `line`, captures the current scope (closure)
`value.arg <index>`	+1	copy of argument `index` (undefined if missing)
`value.args <index>`	+1	array of the arguments from `index` (rest `args...`)
`value.this`	+1	`this` of the current function
`value.variable <name>`	+1	the variable itself (undefined if missing)

Stacks, jumps, errors

Instruction	Effect
`push.args`	open an argument list
`pop.args`	discard the last open list (not the caller's ones)
`push.nsp`	open a local scope
`pop.nsp`	close it and push its `define.return` value
`return`	leave the function (closes its scopes); top level: stop the program
`jump-to <line>`	go to `line`
`jump-if <line>`	pop, go to `line` if true
`jump-else <line>`	pop, go to `line` if false
`namespace <a.b>`	enter a namespace (created if needed), `namespace global` to go back
`try.begin <line>`	start a try block whose catch is at `line`
`try.end`	end the try block (on the normal path)
`throw`	pop and throw the value

Falsy for `jump-*` : `undefined`, `null`, `false`, `0`, `NaN`, `""` .

Operators

Binary: pop `b` , pop `a` , push `a op b` . Unary: pop `a` , push the result. Assignments modify `a` (a variable, member or element) and push it.

Instruction	Op	Instruction	Op
`operator.add`	`a + b`	`operator.assign`	`a = b`
`operator.sub`	`a - b`	`operator.add-assign`	`a += b`
`operator.mul`	`a * b`	`operator.sub-assign`	`a -= b`
`operator.div`	`a / b`	`operator.mul-assign`	`a *= b`
`operator.mod`	`a % b`	`operator.div-assign`	`a /= b`
`operator.equ`	`a == b`	`operator.unary-plus`	`+a`
`operator.dif`	`a != b`	`operator.unary-minus`	`-a`
`operator.great`	`a > b`	`operator.pre-incr`	`++a`
`operator.less`	`a < b`	`operator.post-incr`	`a++`
`operator.great-equ`	`a >= b`	`operator.pre-decr`	`--a`
`operator.less-equ`	`a <= b`	`operator.post-decr`	`a--`
`operator.log-and`	`a && b`	`operator.log-not`	`!a`
`operator.log-or`	`a || b`	`operator.bit-not`	`~a`
`operator.bit-and`	`a & b`	`operator.lshift`	`a << b`
`operator.bit-or`	`a | b`	`operator.rshift`	`a >> b`
`operator.bit-xor`	`a ^ b`	`operator.array`	`a[b]`
`operator.member <name>`	`a.name` (−1 +1)	`operator.call`	`a(list)` (−1 +1)

Arrays & classes

The value is popped and added to the array / class left on the top of the stack.

Instruction	Effect
`array.push-numeric <index>`	`array[index] = value` (copied)
`array.push-literal <name>`	`array.name = value` (copied)
`class.push-unstatic <name>`	instance attribute, or method if a function
`class.push-static <name>`	static member
`class.push-operator <index>`	operator overload (index below)
`class.inherit`	inherit from the popped class
`class.constructor <name>`	set the constructor (pops nothing)

Operator index (`class.push-operator`)

0 `assign` · 1 `unary_plus` · 2 `unary_minus` · 3 `pre_increment` · 4 `post_increment` · 5 `pre_decrement` · 6 `post_decrement` · **7** `add` · 8 `subtract` · 9 `multiply` · 10 `divide` · 11 `modulo` · **12** `equal` · 13 `not_equal` · 14 `greater` · 15 `lesser` · 16 `greater_equal` · 17 `lesser_equal` · 18 `not` · 19 `and` · 20 `or` · 21 `bitwise_not` · 22 `bitwise_and` · 23 `bitwise_or` · 24 `bitwise_xor` · 25 `left_shift` · 26 `right_shift` · 27 `add_assign` · 28 `subtract_assign` · 29 `multiply_assign` · 30 `divide_assign` · **31** `get` · **32** `call`

Pitfalls

- `value.assign` **before** `define.variable` when the value comes from a variable: without it, both names share the number/string (`b++` changes `a`).
- `operator.call` and `value.object` consume the argument list: no `pop.args` after them. Even with no arguments, `push.args` is needed.
- A function body must end with `return` , otherwise execution continues into the next lines. Jump over the body when declaring it.
- After `try.begin` , the catch line receives the error value on the stack: `define.variable e` or `value.pop` .
- An expression statement leaves its result on the stack: end it with `value.pop` .
- `&&` / `||` are plain operators: both operands are evaluated (use `jump-else` for a short-circuit).
- No annotation after `value.string` : it would be part of the string.
- Line numbers shift when lines are inserted: update every jump target.

Running assembly

```
assembly fibonacci.txt  # run, print the result
assembly                # interactive
```

Interactive command	
`file <path>`	run a file
`instructions` / `help`	list the instructions / commands
`exit`	quit

Program `examples/assembly` (built with the examples). `print(...)` is available, an empty line runs the input, line numbers continue between inputs. Full program: `examples/assembly/fibonacci.txt` .

C++ (`LiteScript::Assembly`)	
`Compile(code)`	→ `Result { instructions, error, line }` (`line` 0-based)
`Execute(state, code)`	compile + run, `result.value` = result
`ExecuteAfter(state, code)`	append to the last list: line numbers continue
`Disassemble(instructions)`	instructions → text (e.g. of `Script::Compile`)
`GetErrorMessage(error)`	message of a compile error

Script → assembly

What the compiler generates (`Script::Compile` + `Assembly::Disassemble`).

Variables & calls

```
value.number 1          ; var a = 1 + 2 * 3;
value.number 2
value.number 3
operator.mul
operator.add
value.assign
define.variable a
value.variable a        ; a += 1;
value.number 1
operator.add-assign
value.pop
value.variable print    ; print(a, "!");
push.args
value.variable a
define.arg 0
value.string !
define.arg 1
operator.call
value.pop
```

Script	Assembly
`x = 4;`	`value.variable x` · `value.number 4` · `operator.assign` · `value.pop`
`o.k` / `a[0]`	`value.variable o` · `operator.member k` / `value.variable a` · `value.number 0` · `operator.array`
`o.m()`	`value.variable o` · `operator.member m` · `push.args` · `operator.call` (`this` = `o`)
`[1, { k: 2 }]`	`value.array` · `value.number 1` · `array.push-numeric 0` · `value.array` · `value.number 2` · `array.push-literal k` · `array.push-numeric 1`
`f(g(1))`	`value.variable f` · `push.args` · `value.variable g` · `push.args` · `value.number 1` · `define.arg 0` · `operator.call` · `define.arg 0` · `operator.call` (nested lists)
`return 5;` (top level)	`value.number 5` · `define.return` · `return`

Conditions & loops

```
value.variable a        ; var s = a > 0 ? "+" : "-";
value.number 0
operator.great
jump-else 7
value.string +
jump-to 8
value.string -
value.assign
define.variable s
```

```
value.number 0          ; var i = 0;
define.variable i
value.variable i        ; while (i < 3)
value.number 3
operator.less
jump-else 11            ; exit the loop
value.variable i        ; i++;
operator.post-incr
value.pop
jump-to 3               ; loop
```

- `if / else` : condition, `jump-else <else>` , then-part, `jump-to <end>` , else-part.
- `do … while` : body, condition, `jump-if <body start>` . `break` / `continue` = `jump-to` .
- `switch` : value, then per case `value.copy` · case value · `operator.equ` · `jump-else <next>` ; final `value.pop` .

Functions

```
jump-to 11              ; function add(a, b) {
value.arg 0             ; ← body (line 2)
define.variable a
value.arg 1
define.variable b
value.variable a        ;   return a + b;
value.variable b
operator.add
define.return
return                  ; }
value.callback 2
define.variable add
value.variable add      ; add(2, 3)
push.args
value.number 2
define.arg 0
value.number 3
define.arg 1
operator.call           ; → 5 on the stack
```

Rest parameter `f(a, r...)` : `value.args 1` · `define.variable r` .
Anonymous function: `value.callback <line>` used as a value.

Classes

```
value.class             ; class Point {
value.number 0          ;   var x = 0;
value.assign
class.push-unstatic x
jump-to 14              ;   function Point(x) {
value.arg 0
define.variable x
value.this              ;     this.x = x;
operator.member x
value.variable x
operator.assign
value.pop
return                  ;   }
value.callback 6
class.push-unstatic Point
jump-to 21              ;   function get() {
value.this              ;     return this.x;
operator.member x
define.return
return                  ;   }
value.callback 17
class.push-unstatic get
class.constructor Point ; }
define.variable Point
value.variable Point    ; new Point(4).get()
push.args
value.number 4
define.arg 0
value.object
operator.member get
push.args
operator.call           ; → 4
```

- `class P : B` : `value.class` · `value.variable B` · `class.inherit` first.
- `static var n` : `class.push-static n` . `operator add(o)` : callback then `class.push-operator 7` .

Errors, namespaces, scopes

```
try.begin 6             ; try {
value.string oops
throw                   ;   throw "oops";
try.end                 ; }
jump-to 7
define.variable e       ; catch (e) — error on the stack
```

Script	Assembly
`namespace A.B;` / `namespace global;`	`namespace A.B` / `namespace global`
block with its own variables (manual)	`push.nsp` · … · `define.return` · `pop.nsp` → value pushed, locals gone